

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Курсовая работа по дисциплине

Функциональное программирование

**Система автоматизированного расклада
лекал для минимизации расхода ткани**

Студент,

группы 5130201/40003

_____ Лебедев К. О.

Преподаватель,

к.т.н.

_____ Моторин Д.Е.

«_____» _____ 20__ г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Предметная область и математический аппарат	4
1.1 Задача раскроя и минимизация расхода ткани	4
1.2 Формализация задачи 2D Irregular Packing	5
1.3 No-Fit Polygon	5
1.4 Вычисление NFP через сумму Минковского	6
1.5 Эвристика Bottom-Left Fill	7
2 Исходные данные и их обработка	7
2.1 Источник и формат входных данных	7
2.2 Конвейер обработки изображения	7
2.3 Калибровка в физические единицы	9
3 Описание разработки проекта	10
3.1 Архитектура программного комплекса	10
3.1.1 Модуль Types	10
3.1.2 Модуль Parse.Image	11
3.1.3 Модуль Layout.NFP	11
3.1.4 Модуль Layout.Packing	12
3.1.5 Модули вывода: Output и Render	12
3.1.6 Модули ввода: Input.CLI и Input.FileLoader	13
3.1.7 Модули инфраструктуры: Config и Log	13
3.2 Технические особенности реализации	13
3.2.1 Полигональное представление и системы координат . . .	13
3.2.2 Обработка ошибок через Either	13
3.2.3 Логирование действий и журнал в отчёте	14
3.3 Тестирование	14
4 Примеры работы программы	15
4.1 Загрузка изображения и распознавание лекал	15
4.2 Расчёт расклада	15
4.3 Визуализация раскладки	16
4.4 Обработка ошибок	16
5 Руководство по эксплуатации	17
5.1 Требования к окружению	17
5.2 Запуск программы	17
5.3 Типовой сценарий работы	18
5.4 Обработка ошибок и журналирование	18

Заключение	19
История коммитов проекта	20
Список литературы	22

Введение

При промышленном и ательейном пошиве швейных изделий ткань является одной из основных статей затрат, а её раскрой — этапом, на котором формируется значительная доля отходов. Лекала (выкройки деталей изделия) имеют сложную, как правило невыпуклую форму, и от того, насколько плотно они размещены на полотне, напрямую зависит длина израсходованной ткани. Ручная раскладка трудоёмка и редко близка к оптимальной, поэтому автоматизация этой задачи имеет прямой экономический смысл: даже несколько процентов экономии материала при серийном производстве дают заметный эффект.

Формально задача относится к классу *двумерной упаковки фигур произвольной формы* (2D Irregular Packing) — NP-трудной задаче комбинаторной оптимизации. Точное решение для практически значимого числа деталей неосуществимо за разумное время, поэтому применяются геометрические эвристики. Классическим инструментом служит *No-Fit Polygon* (NFP) — многоугольник, описывающий множество положений одной фигуры относительно другой без их перекрытия; в сочетании с эвристикой *Bottom-Left Fill* (BLF) он позволяет получать плотные раскладки за полиномиальное время.

В настоящей работе разработан консольный программный комплекс на языке Haskell, который принимает растровое изображение разложенных лекал, восстанавливает их полигональное представление, выполняет калибровку в физические единицы (сантиметры) и строит расклад на ткани заданной ширины по методу NFP + BLF с поддержкой поворотов, кратных 90° . Результат сопровождается текстовым отчётом, журналом действий и визуализацией раскладки в формате PNG. Рассматривается полный цикл — от математической постановки задачи до экспериментальной проверки на реальных портновских лекалах.

1 Предметная область и математический аппарат

1.1 Задача раскроя и минимизация расхода ткани

Ткань моделируется как полубесконечная лента (рулон) фиксированной ширины W и неограниченной длины. Каждое лекало представляется замкнутым многоугольником (полигоном) — упорядоченным списком вершин в декартовой системе координат. Начало системы координат каждого лекала помещается в его нижнюю-левую точку, что упрощает позиционирование. Раскладка состоит в размещении всех лекал на ленте без взаимного перекрытия и без выхода за границы ширины W ; минимизируемая величина — итоговая

длина L занятой части ленты. Коэффициент полезного использования ткани

$$\eta = \frac{\sum_i \text{Area}(P_i)}{W \cdot L}$$

служит интегральной оценкой качества расклада: чем плотнее уложены детали, тем выше η .

1.2 Формализация задачи 2D Irregular Packing

Пусть задан набор многоугольников $\{P_1, \dots, P_n\}$ и ширина ленты W . Размещение каждого лекала задаётся парой $(\mathbf{t}_i, \theta_i)$, где $\mathbf{t}_i \in \mathbb{R}^2$ — вектор переноса опорной точки, а $\theta_i \in \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$ — угол поворота. Обозначим через $P_i(\mathbf{t}_i, \theta_i)$ многоугольник P_i , повернутый на θ_i и перенесённый на $\mathbf{t}_i$. Требуется минимизировать

$$L = \max_i \max_{(x,y) \in P_i(\mathbf{t}_i, \theta_i)} y$$

при ограничениях непересечения и попадания в ленту:

$$\text{int } P_i(\mathbf{t}_i, \theta_i) \cap \text{int } P_j(\mathbf{t}_j, \theta_j) = \emptyset \quad (i \neq j), \quad 0 \leq x \leq W \quad \forall (x, y) \in P_i(\mathbf{t}_i, \theta_i).$$

Задача NP-трудна, поэтому используется конструктивная эвристика: лекала размещаются последовательно, и для каждого выбирается «наилучшая» допустимая позиция по фиксированному правилу.

1.3 No-Fit Polygon

Ключевое геометрическое понятие — **No-Fit Polygon**. Для двух многоугольников A (неподвижный) и B (подвижный) NFP определяется так: B совершает скольжение вокруг A , плотно прилегая к его контуру, без вращения; траектория, описываемая выбранной опорной точкой B , образует замкнутый многоугольник $\text{NFP}(A, B)$ (рис. 1). Свойства NFP, используемые в алгоритме:

- если опорная точка B помещена *внутри* $\text{NFP}(A, B)$ — фигуры перекрываются;
- если *вне* — фигуры не имеют общих точек;
- если *на границе* — фигуры касаются, но не перекрываются.

Таким образом, вершины $\text{NFP}(A, B)$ — это позиции, в которых B касается A ; именно они становятся кандидатами на размещение, поскольку плотная упаковка достигается при касании уже размещённых деталей.

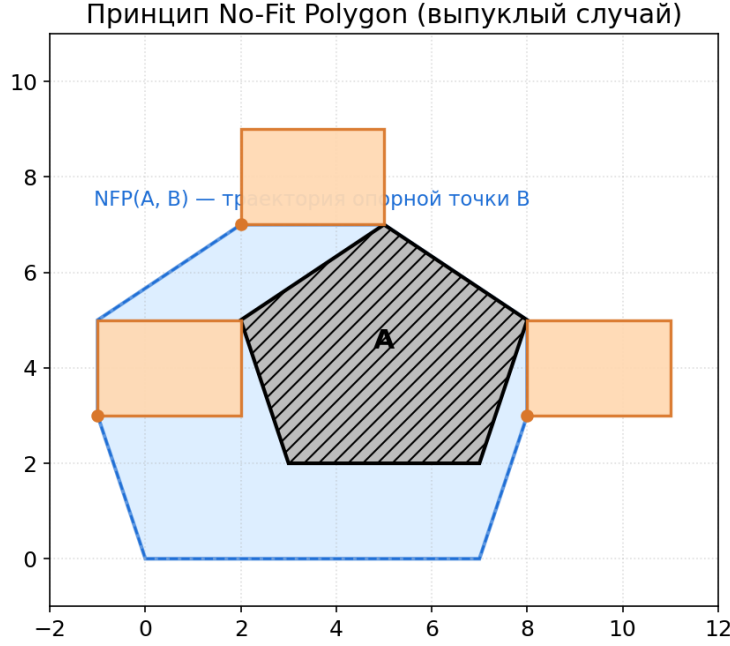


Рис. 1: Принцип No-Fit Polygon: неподвижный полигон A (штриховка), граница $\text{NFP}(A, B)$ (синяя) и подвижный полигон B в трёх касательных позициях.

1.4 Вычисление NFP через сумму Минковского

Для выпуклых многоугольников NFP вычисляется в замкнутой форме как граница суммы Минковского

$$\text{NFP}(A, B) = A \oplus (-B), \quad A \oplus C = \{\mathbf{a} + \mathbf{c} \mid \mathbf{a} \in A, \mathbf{c} \in C\},$$

где $-B = \{-\mathbf{b} \mid \mathbf{b} \in B\}$. Действительно, позиция $\mathbf{p}$ опорной точки B приводит к совпадению некоторой точки $\mathbf{b} \in B$ с точкой $\mathbf{a} \in A$ тогда и только тогда, когда $\mathbf{p} = \mathbf{a} - \mathbf{b} \in A \oplus (-B)$; граница этого множества и есть множество положений касания.

Сумма Минковского двух выпуклых многоугольников, заданных в порядке обхода против часовой стрелки, вычисляется за $O(n + m)$ слиянием их рёбер по полярному углу: рёбра обоих контуров уже отсортированы по углу (следствие выпуклости и CCW-ориентации), поэтому достаточно выполнить слияние, аналогичное шагу сортировки слиянием. Лекала в общем случае невыпуклы; в работе используется консервативная аппроксимация — NFP строится для *выпуклых оболочек* $\text{conv}(A)$ и $\text{conv}(B)$:

$$\widetilde{\text{NFP}}(A, B) = \text{conv}(A) \oplus (-\text{conv}(B)).$$

Такая аппроксимация запрещает часть на самом деле допустимых позиций (в вогнутостях), но *никогда* не допускает перекрытия — это безопасное упрощение. Окончательная проверка валидности каждой позиции выполняется уже по исходным (вогнутым) контурам.

1.5 Эвристика Bottom-Left Fill

После генерации множества кандидатных позиций выбор лучшей выполняется по правилу **Bottom-Left Fill**: среди допустимых позиций выбирается та, у которой минимальна координата Y , а при равенстве Y — минимальна X . Это «прижимает» каждую деталь максимально вниз и влево, минимизируя итоговую длину расклада. Пошагово алгоритм раскладки:

1. отсортировать лекала по убыванию площади (крупные размещаются первыми — это уменьшает фрагментацию свободного пространства);
2. начать с пустого списка размещённых лекал;
3. взять очередное лекало; для каждого разрешённого поворота сгенерировать кандидатные позиции (вершины NFP относительно уже размещённых деталей и точки касания краёв ткани);
4. отфильтровать позиции, при которых лекало выходит за ширину ткани или перекрывает уже размещённые;
5. среди всех валидных позиций и поворотов выбрать лучшую по правилу BLF и зафиксировать размещение;
6. повторять, пока не размещены все лекала.

2 Исходные данные и их обработка

2.1 Источник и формат входных данных

В качестве исходных данных используются открытые портновские лекала, публикуемые на тематических ресурсах [2]. Лекала представлены растровым изображением (PNG/JPEG): набор фигур с чёрным контуром на белом фоне. Программа восстанавливает из изображения полигональное представление каждой фигуры. Растровый ввод выбран как наиболее доступный и универсальный: он не требует векторных исходников и согласуется с приоритетом стандартных средств языка — для чтения изображений используется единственная библиотека `JuicyPixels`, остальная обработка реализована самостоятельно.

2.2 Конвейер обработки изображения

Преобразование раstra в набор полигонов выполняется в четыре этапа (рис. 2).

1. Бинаризация. Изображение переводится в RGB; пиксель считается «чёрным» (принадлежащим контуру), если средняя яркость каналов меньше порога 128. Множество чёрных пикселей сохраняется в `Data.Set` для эффективного поиска принадлежности.

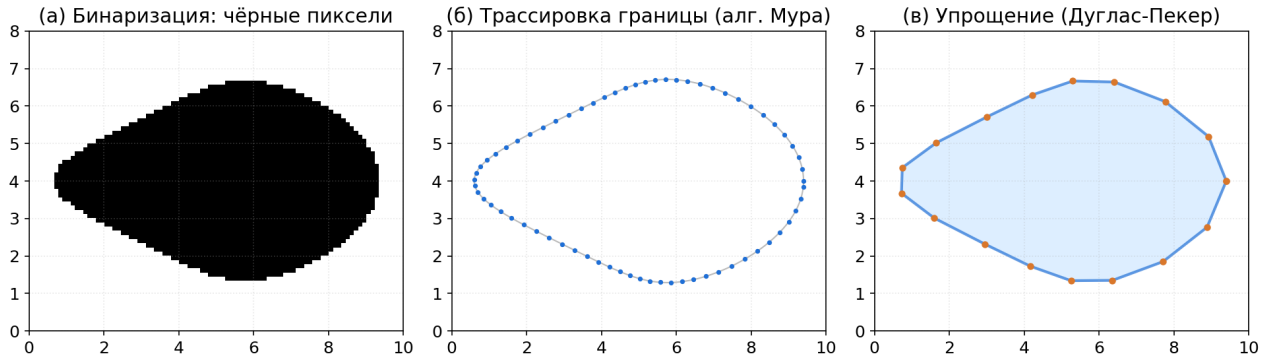


Рис. 2: Конвейер парсера: бинаризация → трассировка границы → упрощение контура.

2. Поиск связанных компонент. Чёрные пиксели разбиваются на связанные компоненты обходом в ширину (BFS) по 8-связности. Каждая компонента соответствует контуру одного лекала. Компоненты меньше порогового размера (80 пикселей) отбрасываются как шум, возникающий из-за артефактов JPEG-сжатия.

3. Трассировка границы. Для каждой компоненты применяется алгоритм трассировки границы Мура (Moore-neighbor tracing). Обход начинается с верхней-левой точки компоненты; на каждом шаге восемь соседей текущего пикселя просматриваются по часовой стрелке, начиная от точки, из которой пришли, до обнаружения следующего граничного пикселя. Результат — упорядоченный по контуру список пикселей (листинг 1). Этот этап принципиально важен: наивное упорядочение точек по углу от центра даёт «зигзаг» для невыпуклых фигур и непригодно для дальнейшего упрощения.

```

1  -- / Трассирует внешнюю границу компоненты, возвращает упорядоченный
2  -- список пикселей контура (алгоритм Мура).
3  mooreTrace :: Set.Set (Int, Int) -> [(Int, Int)]
4  mooreTrace comp
5  | Set.null comp = []
6  | otherwise =
7      let start = minimumBy (comparing \(x, y) -> (y, x)) (Set.toList comp)
8          startB = (fst start - 1, snd start)    -- W сосед: гарантированно фон
9      in start : trace start startB 0
10 where
11     maxSteps = Set.size comp * 8 + 64
12     trace p b n
13     | n >= maxSteps = []
14     | otherwise = case findNext p b of
15         Nothing      -> []
16         Just (nextP, newB)
17         | nextP == startP -> []
18         | otherwise      -> nextP : trace nextP newB (n + 1)
19     startP = minimumBy (comparing \(x, y) -> (y, x)) (Set.toList comp)
20     findNext p b = scan 1 b

```



```

21     where
22         bIdx = dirIndex p b
23         scan i prevBg
24             | i > 8      = Nothing
25             | otherwise =
26                 let idx = (bIdx + i) `mod` 8
27                     cand = addDir p (dirs8 !! idx)
28                 in if Set.member cand comp
29                     then Just (cand, prevBg)
30                     else scan (i + 1) cand

```

Listing 1: Листинг 1: `mooreTrace` — трассировка границы связной компоненты.

4. Упрощение контура. Полученный контур содержит сотни пикселей-вершин. Алгоритм Рамера–Дугласа–Пекера (листинг 2) рекурсивно отбрасывает вершины, отклоняющиеся от хорды менее чем на $\varepsilon = 2$ пикселя, сокращая число вершин в 30–60 раз без потери формы. На завершающем шаге контур переводится в математическую систему координат (ось Y вверх) и нормализуется — сдвигается так, чтобы нижняя-левая точка совпала с началом координат.

```

1  douglasPeucker :: Double -> [Point] -> [Point]
2  douglasPeucker _ [] = []
3  douglasPeucker _ [p] = [p]
4  douglasPeucker _ [p, q] = [p, q]
5  douglasPeucker epsilon pts@(start:_) =
6      let end = last pts
7          (maxDist, maxIdx) = maxPerpIndex start end pts
8      in if maxDist <= epsilon
9          then [start, end]
10         else let left = douglasPeucker epsilon (take (maxIdx + 1) pts)
11                 right = douglasPeucker epsilon (drop maxIdx pts)
12             in case left of
13                 [] -> right
14                 _ -> init left ++ right

```

Listing 2: Листинг 2: `douglasPeucker` — упрощение полилинии.

2.3 Калибровка в физические единицы

Изображение задаёт лекала в пикселях, тогда как раскрой выполняется в сантиметрах. Для калибровки пользователь указывает физическую ширину изображения $W_{\text{см}}$; зная ширину в пикселях $W_{\text{пкс}}$, программа вычисляет масштаб $s = W_{\text{см}}/W_{\text{пкс}}$ (см/пиксель) и умножает все координаты на s . Запрашивается только одна сторона (ширина): высота восстанавливается из пропорций пикселей, что исключает искажение формы при неверно введённом

соотношении сторон. После калибровки вся последующая работа — ширина ткани, расклад, отчёт — ведётся в сантиметрах.

3 Описание разработки проекта

3.1 Архитектура программного комплекса

Проект собирается системой Stack как библиотека `fabric-packer` с исполняемым модулем `fabric-packer` (точка входа `app/Main.hs`) и двумя отладочными исполняемыми модулями `test-parser` и `test-pack`. Вся логика сосредоточена в библиотеке, что позволяет тестировать компоненты независимо от точки входа. Исходный код организован по слоям: типы данных, парсинг, геометрия и расклад, вывод, ввод и интерфейс, инфраструктура (конфигурация и логирование).

3.1.1 Модуль Types

`Types` — центральный модуль, не зависящий ни от одного другого и определяющий всю систему типов. Геометрия задана синонимами `Point = (Double, Double)`, `Polygon = [Point]`, `Angle = Double`. Лекало `Piece` несёт порядковый номер и полигон; ткань `Fabric` — ширину; `PlacedPiece` — размещённое лекало (полигон, позиция, угол); `Layout` — итоговую раскладку. Конфигурация `AppConfig` снабжена выводимыми экземплярами `FromJSON/ToJSON` (через `GHC.Generics`) для сериализации в JSON. Иерархия ошибок `AppError` разделяет четыре класса нарушений (листинг 3).

```
1 data Piece = Piece { pieceId :: Int, piecePolygon :: Polygon }
2   deriving (Show, Eq)
3
4 data Fabric = Fabric { fabricWidth :: Double } deriving (Show, Eq)
5
6 data PlacedPiece = PlacedPiece
7   { ppPiece :: Piece, ppPosition :: Point, ppRotation :: Angle }
8   deriving (Show, Eq)
9
10 data Layout = Layout
11   { layFabric :: Fabric, layPlaced :: [PlacedPiece], layLength :: Double }
12   deriving (Show, Eq)
13
14 data AppError = ParseErr String | IOErr String
15               | ValidationErr String | LayoutErr String
16   deriving (Show, Eq)
```

Listing 3: Листинг 3: основные типы данных проекта.

3.1.2 Модуль Parse.Image

`Parse.Image` реализует описанный конвейер. Функция `loadPieces` читает изображение, бинаризует его, выделяет связные компоненты, трассирует и упрощает контуры, возвращая список лекал и ширину изображения в пикселях. Функция `scalePieces` применяет калибровочный масштаб. Сигнатура подчёркивает явную обработку ошибок: результат обернут в `Either AppError`.

```
1 loadPieces :: FilePath -> IO (Either AppError ([Piece], Int))
2 scalePieces :: Double -> [Piece] -> [Piece]
```

Listing 4: Листинг 4: публичный интерфейс модуля парсинга.

3.1.3 Модуль Layout.NFP

`Layout.NFP` содержит геометрические примитивы: вычисление выпуклой оболочки (алгоритм Эндрю, $O(n \log n)$), сумму Минковского выпуклых полигонов, построение NFP и проверку пересечения двух произвольных полигонов. Сумма Минковского реализована слиянием рёбер по полярному углу (листинг 5).

```
1 minkowskiSum :: Polygon -> Polygon -> Polygon
2 minkowskiSum p q =
3   case (rotateToBottommost (orientCCW p), rotateToBottommost (orientCCW q)) of
4     (p0:_pTail, q0:_qTail) | length p >= 2 && length q >= 2 ->
5       let edsP = polyEdges (p0 : _pTail)
6           edsQ = polyEdges (q0 : _qTail)
7           eds  = mergeByAngle edsP edsQ
8           start = p0 `addP` q0
9       in scanl addP start (init eds)
10  _ -> []
```

Listing 5: Листинг 5: `minkowskiSum` — сумма Минковского двух выпуклых CCW-полигонов.

Проверка пересечения `polygonsOverlap` работает для произвольных (в т.ч. невыпуклых) полигонов и использует трёхступенчатую схему: быстрое отсечение по ограничивающим прямоугольникам, проверка принадлежности вершин одного полигона другому (метод трассировки луча) и проверка пересечения рёбер. Касание не считается перекрытием, что допускает плотное прилегание деталей.

3.1.4 Модуль Layout.Packing

Layout.Packing реализует основной алгоритм NFP + BLF (листинг 6). Функция `packPatterns` сортирует лекала по убыванию площади и сворачивает (`foldl`) список, последовательно размещая каждую деталь. Для каждого лекала `bestPlacement` перебирает четыре поворота, для каждого собирает кандидатные позиции (вершины NFP относительно размещённых деталей, их проекции на края ткани и угол $(0,0)$), фильтрует их по непересечению и попаданию в ширину, и выбирает лучшую по правилу BLF. Если деталь не помещается ни в одной ориентации, она пропускается, а пользователь получает предупреждение.

```
1 packPatterns :: Fabric -> [Piece] -> Layout
2 packPatterns fab pieces =
3   let sorted = sortByAreaDesc pieces
4       placed = foldl (placeOne fab) [] sorted
5   in Layout fab placed (totalLength placed)
6
7 bestPlacement :: Fabric -> [PlacedPiece] -> Piece -> Maybe PlacedPiece
8 bestPlacement fab placed piece =
9   let attempts =
10      [ (rot, pos)
11        | rot <- allowedRotations
12          , let poly = preparePoly rot (piecePolygon piece)
13            , let (pw, _) = bboxSize poly
14              , pw <= fabricWidth fab
15              , pos <- maybeToList (bestBLFPosition fab placed poly pw)
16        ]
17   in case attempts of
18      [] -> Nothing
19      xs -> let (rot, pos) = minimumBy (comparing snd) xs
20             in Just (PlacedPiece piece pos rot)
```

Listing 6: Листинг 6: ядро алгоритма расклада NFP + BLF.

3.1.5 Модули вывода: Output и Render

Output отвечает за текстовый вывод: `printPiecesInfo` печатает сводку о распознанных лекалах (число вершин, площадь, габариты), `printLayout` — результат расклада, `printError` — человекочитаемые сообщения об ошибках, `writeReport` — сохранение отчёта в текстовый файл вместе с журналом действий сессии. Render формирует визуализацию раскладки: контуры размещённых лекал растеризуются алгоритмом Брезенхема и записываются в PNG (белый фон, чёрные контуры); разрешение вывода задаётся в пикселях на сантиметр, что сохраняет масштаб исходного изображения.

3.1.6 Модули ввода: `Input.CLI` и `Input.FileLoader`

`Input.FileLoader` инкапсулирует чтение файлов с диска с проверкой существования. `Input.CLI` реализует интерактивное текстовое меню и хранит состояние сессии `AppState` (текущие лекала, масштаб, ширина ткани, последний расклад, журнал). Главный цикл `loop` печатает меню, считывает действие, логирует его и диспетчеризует в `runAction`. Каждый ввод чисел защищён функциями `promptDouble/promptInt`, повторно запрашивающими значение при некорректном вводе.

3.1.7 Модули инфраструктуры: `Config` и `Log`

`Config` читает и записывает конфигурацию в файл `fabric-packer.json` (через `aeson`); при отсутствии файла `loadConfig` возвращает ошибку `IOErr`, а приложение стартует с конфигурацией по умолчанию. `Log` записывает события в `app.log` с временными метками и возвращает отформатированную строку, что позволяет одновременно накапливать журнал сессии для включения в отчёт.

3.2 Технические особенности реализации

3.2.1 Полигональное представление и системы координат

В программе сосуществуют две системы координат: экранная (ось Y направлена вниз, используется при чтении растра) и математическая (ось Y вверх, используется в геометрии и раскладе). Преобразование `toMathPoint` выполняется один раз на границе модуля парсинга, после чего весь остальной код оперирует единообразными математическими координатами. Все лекала нормализованы — их нижняя-левая точка совпадает с началом координат, что соответствует требованию ТЗ и упрощает позиционирование.

3.2.2 Обработка ошибок через `Either`

Все операции, способные завершиться неуспехом (чтение файла, парсинг изображения, парсинг конфигурации, валидация ввода), возвращают `Either AppError`. Это исключает аварийное завершение программы: ошибка является обычным значением, которое распространяется по типу и обрабатывается в одной точке — функции `recordError`, печатающей сообщение пользователю и фиксирующей его в журнале. Тип `AppError` разделяет ошибки парсинга, ввода-вывода, валидации и расклада, что позволяет формировать осмысленные диагностические сообщения.

3.2.3 Логирование действий и журнал в отчёте

Функция `logEvent` одновременно дописывает событие в `app.log` и возвращает отформатированную строку. Состояние сессии `AppState` накапливает эти строки в поле `stLog`; при сохранении отчёта журнал целиком включается в `report.txt` отдельным разделом. Логируются все действия пользователя, результаты (число распознанных лекал, размещённых деталей, итоговая длина) и ошибки с временными метками, что обеспечивает прослеживаемость работы.

3.3 Тестирование

Корректность ключевых компонентов подтверждена тестами свойств на основе библиотеки `QuickCheck`: для каждого свойства генерируются случайные входные данные, и проверяется выполнение инварианта. Покрыты следующие группы свойств.

- **Геометрические инварианты.** Площадь полигона инвариантна относительно поворота на углы, кратные 90° , и относительно переноса: $\text{Area}(R_\theta P) = \text{Area}(P)$. Нормализация полигона даёт неотрицательные координаты, а её нижняя-левая точка совпадает с началом координат.
- **Сумма Минковского и NFP.** Число вершин суммы Минковского двух выпуклых полигонов не превосходит суммы чисел их вершин; результат `convexHull` выпукл (все повороты вдоль контура одного знака).
- **Проверка пересечения.** Полигон всегда пересекается сам с собой; два полигона с непересекающимися ограничивающими прямоугольниками не перекрываются; `polygonsOverlap` симметрична.
- **Свойства расклада.** Для любой ширины ткани все размещённые лекала попадают в полосу $[0, W]$ по оси X и в полуплоскость $Y \geq 0$; никакие два размещённых лекала не перекрываются; число размещённых деталей не превосходит числа поданных.
- **Упрощение контура.** Дуглас-Пекер не увеличивает число вершин и сохраняет крайние точки полилинии.

Пример свойства, проверяющего отсутствие перекрытий в построенном раскладе, приведён в листинге 7.

```
1 prop_noOverlap :: Positive Double -> [Piece] -> Bool
2 prop_noOverlap (Positive w) pieces =
3   let placed = layPlaced (packPatterns (Fabric w) pieces)
4       polys  = map placedWorldPolygon placed
5   in and [ not (polygonsOverlap a b)
6           | (i, a) <- zip [0..] polys
7           , (j, b) <- zip [0..] polys
8           , i < (j :: Int) ]
```

Listing 7: Листинг 7: свойство QuickCheck — построенный расклад не содержит перекрытий.

4 Примеры работы программы

4.1 Загрузка изображения и распознавание лекал

После запуска программа отображает текстовое меню. По выбору пункта 1 пользователь указывает путь к изображению и его физическую ширину; программа распознаёт лекала и печатает сводку. На тестовом изображении портновских лекал (рис. 3) при ширине 40 см распознано 9 деталей; фрагмент вывода:

Ширина изображения: 1093 пкс

Физическая ширина изображения (см): 40

Найдено лекал: 9

```
-----  
Лекало #1 | вершин: 12 | площадь: 7 см2 | размер: 5×3 см  
Лекало #2 | вершин: 27 | площадь: 70 см2 | размер: 8×12 см  
Лекало #3 | вершин: 23 | площадь: 16 см2 | размер: 9×4 см  
Лекало #4 | вершин: 13 | площадь: 7 см2 | размер: 5×3 см  
Лекало #5 | вершин: 24 | площадь: 70 см2 | размер: 8×12 см  
Лекало #6 | вершин: 22 | площадь: 125 см2 | размер: 12×13 см  
Лекало #7 | вершин: 41 | площадь: 381 см2 | размер: 37×16 см  
Лекало #8 | вершин: 21 | площадь: 124 см2 | размер: 12×13 см  
Лекало #9 | вершин: 38 | площадь: 209 см2 | размер: 23×14 см  
-----
```

4.2 Расчёт расклада

По выбору пункта 2 задаётся ширина ткани, по пункту 3 запускается расчёт. При ширине 160 см все 9 лекал размещены, итоговая длина расклада — 28 см. Фрагмент вывода:

Ширина ткани : 160 см

Длина расклада: 28 см

Размещено лекал: 9

```
Лекало #7 позиция (0, 0)    угол 0 град.  
Лекало #9 позиция (137, 0)  угол 0 град.  
Лекало #6 позиция (0, 14)   угол 180 град.  
...
```

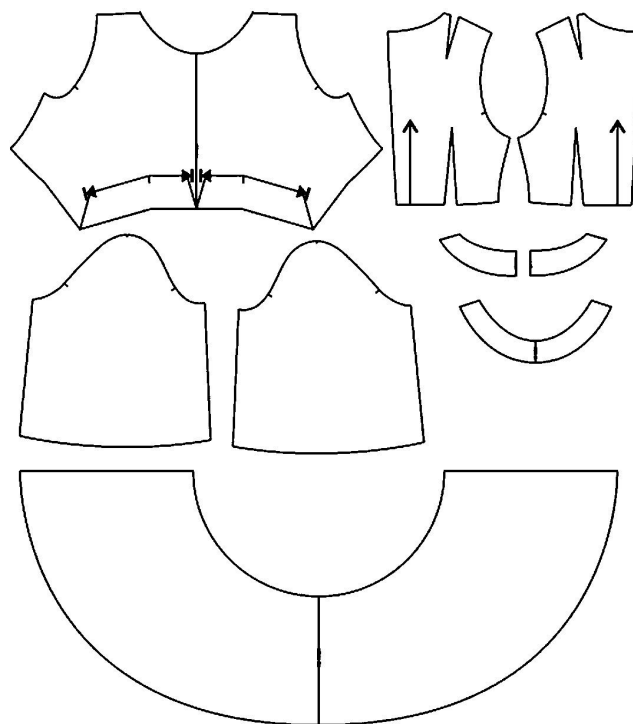


Рис. 3: Исходное изображение: портновские лекала (чёрный контур на белом фоне).

4.3 Визуализация раскладки

Пункт 5 сохраняет визуализацию в `layout.png` (рис. 4): белый фон, чёрные контуры размещённых лекал; ось X — ширина ткани, ось Y — длина. Изображение позволяет визуально оценить плотность укладки и корректность поворотов.

4.4 Обработка ошибок

Программа корректно реагирует на нештатные ситуации без аварийного завершения. При указании несуществующего файла выводится **Ошибка: Парсинг: Не удалось открыть изображение: eger: withBinaryFile: does not exist (No such file or directory)**; при отрицательной или нечисловой ширине ввод запрашивается повторно либо выводится **Ошибка: Валидация: Ширина ткани должна быть положительной или Ошибка: Введите число.**; при попытке расчёта без загруженных лекал — соответствующая подсказка. Все ошибки фиксируются в журнале и попадают в отчёт. Пример фрагмента журнала из `report.txt`:

```
=== Журнал действий и ошибок ===
[2026-05-22 02:18:42] [Info] Выбрано действие: расчёт расклада
[2026-05-22 02:18:42] [Error] Ошибка валидации: Сначала загрузите изображение лекал
[2026-05-22 02:18:43] [Info] Загружено лекал: 9 из "data/testing.png", ширина 40 см
[2026-05-22 02:18:43] [Info] Ширина ткани задана: 160 см
[2026-05-22 02:18:43] [Info] Расклад выполнен: размещено 9 из 9 лекал, длина 28 см
```

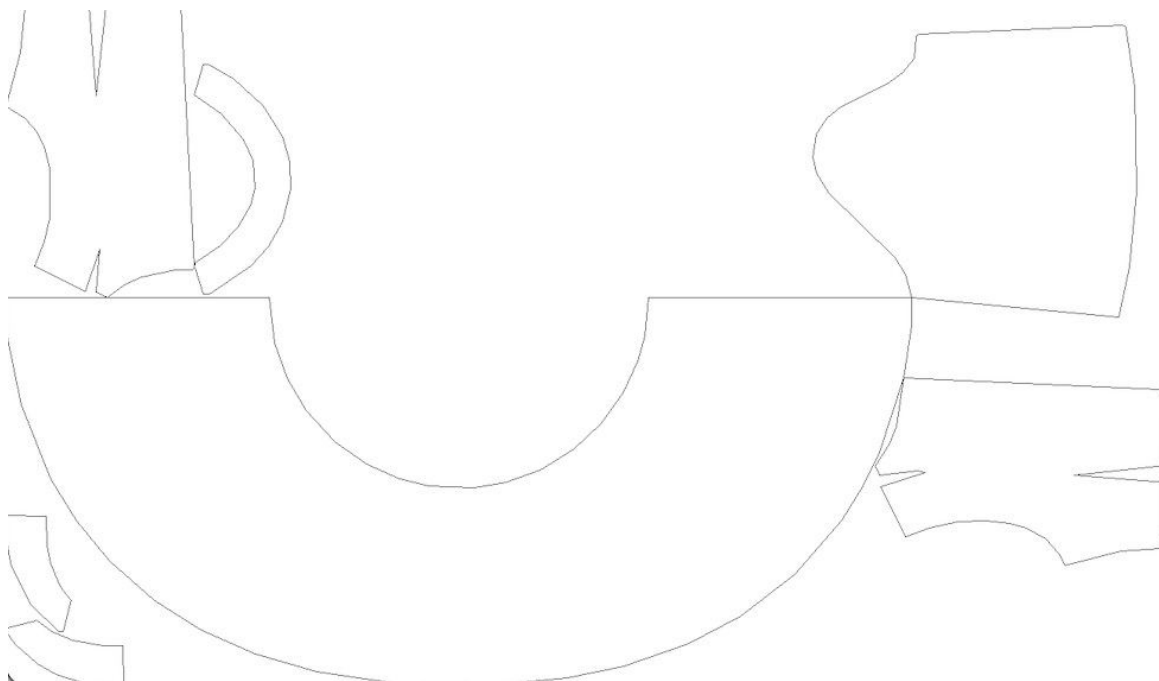



Рис. 4: Фрагмент результата расклада: размещение 9 лекал на ткани шириной 160 см.

5 Руководство по эксплуатации

5.1 Требования к окружению

Для сборки и запуска необходимы GHC и система сборки Stack [3]. После клонирования репозитория команда `stack build` автоматически загружает требуемую версию компилятора и все зависимости (`JuicyPixels`, `aeson`, `containers`, `directory`, `time`). Входные данные — изображение PNG/JPEG с фигурами в виде чёрного контура на белом фоне.

5.2 Запуск программы

Программа запускается из корневой директории проекта командой `stack run` (или `stack exec fabric-packer`). Отображается главное меню:

```
=== Fabric Packer ===
```

1. Загрузить изображение лекал
2. Задать ширину ткани
3. Рассчитать расклад
4. Сохранить отчёт
5. Сохранить изображение расклада (PNG)
6. Загрузить конфиг
7. Сохранить конфиг
0. Выход

5.3 Типовой сценарий работы

1. Пункт 1 — указать путь к изображению (например, `data/testing.png`) и физическую ширину изображения в сантиметрах. Программа распознаёт лекала и печатает их сводку.
2. Пункт 2 — задать ширину ткани в сантиметрах.
3. Пункт 3 — выполнить расчёт расклада; выводятся длина расклада и позиции деталей.
4. Пункт 4 — сохранить текстовый отчёт (`report.txt`) с журналом сессии.
5. Пункт 5 — сохранить визуализацию расклада (`layout.png`).
6. Пункты 6–7 — загрузить/сохранить конфигурацию (пути вывода и ширину ткани по умолчанию) в `fabric-packer.json`.

5.4 Обработка ошибок и журналирование

При отсутствии или повреждении файла, некорректном вводе и невозможности разместить деталь программа выводит диагностическое сообщение и продолжает работу. Все действия и ошибки записываются в `app.log` с временными метками; тот же журнал включается в текстовый отчёт `report.txt`.

Заключение

В ходе работы реализован консольный программный комплекс на языке Haskell для автоматизированного расклада швейных лекал с целью минимизации расхода ткани. Реализация выполнена преимущественно стандартными средствами языка; единственная внешняя зависимость предметной области — библиотека `JuicyPixels` для чтения растровых изображений.

Разработан полный конвейер обработки: восстановление полигонального представления лекал из растрового изображения (бинаризация, поиск связанных компонент, трассировка границы алгоритмом Мура, упрощение Дугласа–Пекера), калибровка в физические единицы, и алгоритм двумерной упаковки фигур произвольной формы на основе No-Fit Polygon и эвристики Bottom-Left Fill с поддержкой поворотов, кратных 90° . NFP вычисляется через сумму Минковского выпуклых оболочек с последующей точной проверкой пересечения исходных вогнутых контуров. Обеспечены консольный интерфейс, чтение и запись конфигурации в JSON, текстовый отчёт, визуализация расклада в PNG, логирование действий и ошибок, а также устойчивая обработка ошибок без аварийного завершения. Корректность ключевых компонентов подтверждена тестами свойств на основе QuickCheck.

На тестовом наборе из 9 портновских лекал при ширине ткани 160 см получен расклад длиной 28 см с коэффициентом использования ткани $\approx 0,61$; время расчёта — менее секунды. Дальнейшее повышение плотности укладки возможно за счёт перехода от выпуклой аппроксимации NFP к точному NFP для невыпуклых полигонов и добавления поворотов на произвольные углы.

Сравнение плановых и фактических сроков выполнения этапов приведено на рис. 5. Основное отклонение от плана связано с затянувшимся согласованием технического задания, что сдвинуло старт реализации; после фиксации требований работа велась в плановом темпе.

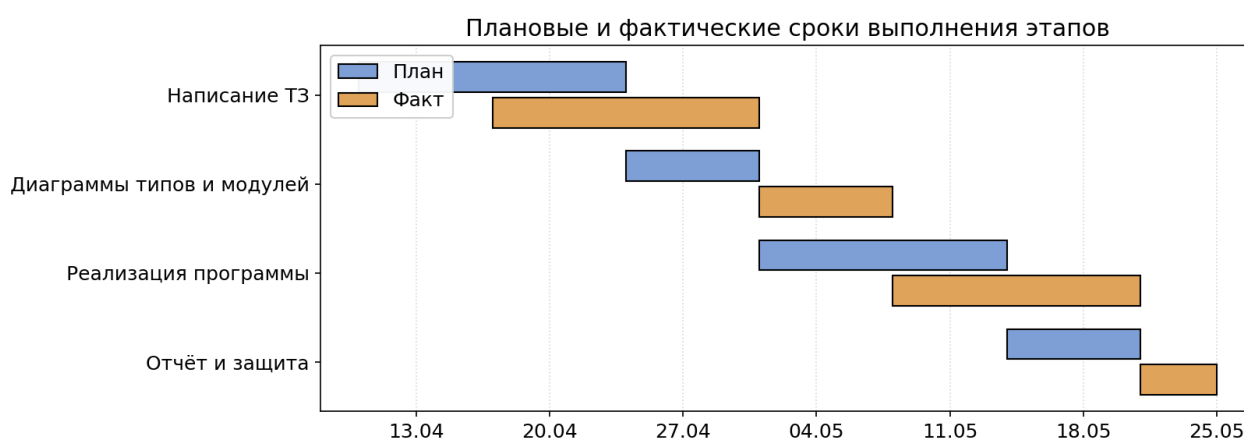


Рис. 5: Сравнение плановых и фактических сроков выполнения этапов.

История коммитов проекта

Ниже приведены коммиты, отражающие ход разработки программной реализации. Указаны короткий хэш, дата и время с точностью до минут и подробное описание содержания.

- d973cfa (22.05.2026, 05:31).

Сообщение: *init Stack project scaffolding for fabric-packer.*

Создан каркас Stack-проекта **fabric-packer**: файлы `stack.yaml`, `fabric-packer.cabal`, `Setup.hs`, `LICENSE`, `README.md`, `hie.yaml`. Настроена структура библиотеки и исполняемого модуля, объявлены зависимости (`base`, `JuicyPixels`, `aeson`, `containers`, `directory`, `time`), добавлен `.gitignore`. Зафиксирована успешная сборка пустого проекта.

- 211355f (22.05.2026, 05:44).

Сообщение: *add core types, logging, JSON config and image parser.*

Реализован модуль **Types** со всей системой типов (геометрия, лекала, ткань, раскладка, конфигурация, иерархия ошибок **AppError**, уровни логирования). Добавлены модуль **Log** (запись событий с временными метками) и **Config** (чтение/запись JSON через `aeson`). Реализован модуль **Parse.Image**: бинаризация, поиск связанных компонент (BFS, 8-связность), трассировка границы алгоритмом Мура, упрощение Дугласа–Пекера, перевод в математические координаты и нормализация.

- 49afa30 (22.05.2026, 10:19).

Сообщение: *add NFP+BLF packing algorithm, output and PNG rendering.*

Реализован модуль **Layout.NFP**: выпуклая оболочка (алгоритм Эндрю), сумма Минковского выпуклых полигонов слиянием рёбер по углу, построение NFP и проверка пересечения произвольных полигонов. Реализован модуль **Layout.Packing**: сортировка по площади, перебор поворотов, генерация кандидатных позиций по NFP, фильтрация и выбор по правилу BLF. Добавлены модули **Output** (текстовый вывод и отчёт) и **Render** (растеризация контуров алгоритмом Брезенхема и запись PNG).

- b0be7c1 (22.05.2026, 11:41).

Сообщение: *add CLI, entry point, dev executables and sample data.*

Реализован модуль **Input.CLI** (интерактивное меню, состояние сессии, диспетчеризация действий, валидация ввода) и **Input.FileLoader**. Добавлена точка входа `app/Main.hs`, отладочные исполняемые модули `test-parser` и `test-pack`, а также тестовое изображение лекал. Собрана работоспособная сквозная версия: загрузка изображения → расклад → вывод и визуализация.

- f3a9c0d (22.05.2026, 19:14).

Сообщение: *add cfgFabricWidth to config, fix LoadConfig in CLI, add QuickCheck*

tests.

В конфигурацию добавлено поле ширины ткани по умолчанию `cfgFabricWidth`; пункт меню «Загрузить конфиг» корректно восстанавливает ширину ткани из файла. Реализована калибровка изображения в сантиметры. Добавлен набор тестов свойств на основе QuickCheck (геометрические инварианты, свойства суммы Минковского, проверки пересечения, инварианты расклада, упрощение контура). Зафиксирована финальная стабильная версия.

Список литературы

- [1] Two-dimensional irregular packing problems: A review // Frontiers in Mechanical Engineering. — 2022. — Vol. 8. — Art. 966691. — URL: <https://www.frontiersin.org/journals/mechanical-engineering/articles/10.3389/fmech.2022.966691/full> (дата обращения: 25.05.2026).
- [2] Портновский блог: выкройки и лекала // Portnoyblog. — URL: <https://portnoyblog.com/> (дата обращения: 25.05.2026).
- [3] The Haskell Tool Stack. — URL: <https://docs.haskellstack.org> (дата обращения: 25.05.2026).
- [4] JuicyPixels: Haskell library to load and serialize images // Hackage. — URL: <https://hackage.haskell.org/package/JuicyPixels> (дата обращения: 25.05.2026).
- [5] Douglas D. H., Peucker T. K. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature // Cartographica. — 1973. —
- [6] Gonzalez R. C., Woods R. E. Digital Image Processing. — 3rd ed. — Pearson, 2008.